

Week 5

Local Storage & Offline First

Pemrograman Mobile

Flutter • SharedPreferences • SQLite • Offline-First

Dari aplikasi yang butuh internet → aplikasi yang tetap berguna saat offline

Learning Outcomes

Setelah mengikuti pertemuan ini, mahasiswa mampu:

1. Menjelaskan perbedaan storage key-value, relasional, dan NoSQL di perangkat.
2. Menyimpan preferensi sederhana dengan `SharedPreferences`.
3. Menerapkan CRUD catatan dengan SQLite (`sqflite`) via repository lokal.
4. Menerapkan pola offline-first: cache-first, dirty flag, antrean sync.
5. Menangani loading, data, error, dan empty state untuk data lokal.
6. Menguji repository lokal dengan repository palsu.
7. Membandingkan pilihan storage usulan AI dan mempertahankannya.

Mini Poll

Jika user menulis catatan di mode pesawat, lalu menutup aplikasi...

Apa yang seharusnya terjadi?

- A. Catatan hilang, tidak apa-apa
- B. Aplikasi crash
- C. Catatan tersimpan lokal & terkirim saat online
- D. Minta AI mengembalikan catatan

Diskusi

Apa masalah jika daftar 1000 catatan disimpan sebagai satu string JSON di SharedPreferences?

Bagian 1

Jenis Storage Lokal

Pilihan Storage

Aturan praktis:

Kebutuhan	Pilihan	Contoh
Pengaturan kecil key-value	SharedPreferences	tema, bahasa
Data relasional	SQLite via sqflite	catatan, tugas
NoSQL ringan	Hive	cache objek
Relasional reaktif	Drift	app besar + stream

Week 5 memakai:

SharedPreferences (preferensi) + SQLite (catatan)

Apa Itu SharedPreferences?

Penyimpanan key-value kecil dan primitif.

```
final prefs =  
    await SharedPreferences.getInstance();  
  
await prefs.setBool('dark_mode', true);  
final dark =  
    prefs.getBool('dark_mode') ?? false;
```

Cocok untuk:

```
dark_mode      → bool  
language       → String  
last_opened_at → String
```

Hanya nilai kecil. Bukan database.

Kapan SharedPreferences Tidak Cukup?

Daftar catatan ≠ pengaturan.

```
SharedPreferences:  
  "notes" → "[{...1000 item...}]" ← rapuh!
```

Masalah:

- update 1 catatan = tulis ulang semua
- tidak bisa query / sort / filter
- sync parsial mustahil

Aturan:

- koleksi data → selalu database
- preferensi → SharedPreferences

Pertanyaan

Apa yang rusak jika 1000 catatan disimpan sebagai satu string JSON?

Bagian 2

SQLite & Repository Lokal

Membuka Database

Satu fungsi pembuka untuk seluruh repository:

```
Future<Database> openNotesDb() async {  
  final dir = await getDatabasesPath();  
  return openDatabase(  
    p.join(dir, 'offline_notes.db'),  
    version: 1,  
    onCreate: (db, version) async {  
      await db.execute('''  
        CREATE TABLE notes(  
          id INTEGER PRIMARY KEY AUTOINCREMENT,  
          title TEXT NOT NULL,  
          body TEXT NOT NULL DEFAULT '',  
          updated_at TEXT NOT NULL,  
          dirty INTEGER NOT NULL DEFAULT 0  
        )  
      ''');  
    },  
  );  
}
```

Model Note

Field `dirty` menandai catatan belum tersinkron:

```
class Note {  
    final int? id;  
    final String title;  
    final String body;  
    final DateTime updatedAt;  
    final bool dirty;  
  
    Map<String, Object?> toMap() => {  
        'id': id,  
        'title': title,  
        'body': body,  
        'updated_at':  
            updatedAt.toIso8601String(),  
        'dirty': dirty ? 1 : 0,  
    };  
}
```

Mapping hanya terjadi di model, bukan di UI.

Null Safety pada fromMap

Database nyata bisa berisi baris tak lengkap.

```
factory Note.fromMap(  
    Map<String, Object?> map,  
) {  
    return Note(  
        id: (map['id'] as num?)?.toInt(),  
        title: map['title'] as String? ?? '',  
        body: map['body'] as String? ?? '',  
        updatedAt: DateTime.tryParse(  
            map['updated_at'] as String? ?? '',  
        ) ?? DateTime.fromMillisecondsSinceEpoch(0),  
        dirty:  
            ((map['dirty'] as num?)?.toInt()  
             ?? 0) == 1,  
    );  
}
```

Aturan:

- gunakan `as String?`, bukan `as String`
- sediakan default yang masuk akal

Repository Lokal

Repository memisahkan UI dari database.

```
UI (Widget)
  ↓ memanggil
Provider (Riverpod)
  ↓ memakai
NoteRepository
  ↓ memakai
sqlite / SharedPreferences
```

UI tidak tahu:

- nama file database
- SQL
- SharedPreferences

UI hanya tahu:

```
Future<List<Note>> fetchNotes()  
Future<Note> addNote(title: ...)  
Future<void> deleteNote(id)
```

Contoh Repository

```
class NoteRepository {  
    NoteRepository({Future<Database> Function()? openDb})  
        : _openDb = openDb ?? openNotesDb;  
  
    final Future<Database> Function() _openDb;  
  
    Future<List<Note>> fetchNotes() async {  
        final db = await _openDb();  
        final rows = await db.query(  
            'notes',  
            orderBy: 'updated_at DESC',  
        );  
        return rows.map(Note.fromMap).toList();  
    }  
}
```

Constructor menerima `openDb` agar test bisa menyuntikkan database palsu.

Keuntungan Repository Lokal:

- query SQL terpusat
- mudah diganti sumber data
- mudah di-mock untuk testing

Contoh Repository + Riverpod

```
final noteRepositoryProvider =  
    Provider<NoteRepository>(  
        (ref) => NoteRepository(),  
    );  
  
final notesProvider =  
    FutureProvider<List<Note>>(  
        (ref) => ref  
            .watch(noteRepositoryProvider)  
            .fetchNotes(),  
    );
```

Widget → watch(notesProvider) → Repository → SQLite

Setelah mutasi data:

```
ref.invalidate(notesProvider);
```

Bagian 3

Offline-First: Cache & Sync

Tiga Mekanisme Inti

1. Cache-first read
tampilkan lokal seketika
→ refresh background → simpan
2. Dirty flag
perubahan belum terkirim → dirty = 1
3. Antrean sinkronisasi
proses berurutan saat online
→ tandai bersih bila server 2xx

UI → Provider → Repository → SQLite
 ↓ sync
 REST API

Cache-First Read

Jangan biarkan UI blank saat offline.

```
Future<List<Post>> loadPostsCacheFirst()
    async {
        final cached =
            await readCachedPosts();
        // 1. kembalikan cache segera
        // 2. background: fetch Dio
        //    → simpan → invalidate
        refreshPostsInBackground();
        return cached;
    }
```

Week 4 memberi API-nya, Week 5 memberi cache-nya.

Sinkronisasi Dirty Notes

Server disimulasikan, mekanismenya yang dinilai:

```
Future<int> syncNotes(  
    NoteRepository repo,  
) async {  
    final dirtyCount =  
        await repo.countDirty();  
    if (dirtyCount == 0) return 0;  
  
    // project nyata: kirim tiap catatan  
    // dirty ke REST API di sini  
    await Future.delayed(  
        const Duration(seconds: 1),  
    );  
    await repo.markAllSynced();  
    return dirtyCount;  
}
```

Aturan Konflik

Tanpa aturan eksplisit, sync menimpa data diam-diam.

Pilihan umum:

```
last-write-wins → updated_at terbaru menang
server-wins     → server selalu benar
client-wins     → lokal selalu benar
manual-merge    → user memilih
```

Week 5 memakai:

```
| last-write-wins berdasarkan updated_at
```

Diskusi

```
| Kapan last-write-wins berbahaya? (contoh: saldo, stok)
```

UI State untuk Data Lokal

Sama seperti Week 4, data lokal pun punya 4 state:

```
return state.when(  
  loading: () => const Center(  
    child:  
      CircularProgressIndicator(),  
  ),  
  data: (notes) {  
    if (notes.isEmpty) {  
      return const Center(  
        child: Text(  
          'Belum ada catatan.',  
        ),  
      );  
    }  
    return NotesList(notes);  
  },  
  error: (e, _) => ErrorView(  
    message: '$e',  
    onRetry: () => ref.invalidate(  
      notesProvider,  
    ),  
  ),  
);
```

Tambahan khas offline: badge jumlah `dirty` .

Simulasi Offline Deterministik

Jangan bergantung pada Wi-Fi kelas.

1. Aktifkan mode pesawat
2. Buka aplikasi → catatan tetap tampil
3. Tambah catatan → badge dirty bertambah
4. Matikan mode pesawat → jalankan sync
5. Badge kembali ke 0

Plus toggle `forceOffline` di provider:

Demo dan testing tidak bergantung kondisi jaringan.

Studi Kasus

Offline Notes

```
NotesPage
  ↓ watch(notesProvider)
  NoteRepository.fetchNotes()
  ↓ query SQLite
  List<Note> → UI + badge dirty
```

Anti-Pattern

Hindari:

1. Koleksi di SharedPreferences

```
simpan 1000 notes = 1 string raksasa
```

2. Query SQL di UI

Semua `db.query` harusnya di repository.

3. Lupa invalidate provider

```
// setelah add/delete:  
ref.invalidate(provider);
```

4. Test menyentuh database sungguhan

Pakai repository palsu via override.

AI Lab

Gunakan AI sebagai Co-Developer

Contoh prompt:

Aplikasi Flutter Offline Notes: CRUD catatan + preferensi tema.

Bandingkan SharedPreferences, Hive, sqflite, dan Drift:

- kompleksitas query, relasi, reaktivitas
- type-safety, boilerplate, testability

Beri rekomendasi final dalam 1 tabel + skema untuk 1000+ catatan. Jelaskan trade-off-nya.

Jangan berhenti pada output AI.

AI Verification Challenge

Setelah AI menghasilkan rekomendasi:

1. **Baca** — pahami tiap opsi storage.
2. **Verifikasi** — coba `flutter pub add` + skema-nya.
3. **Tolak/terima** — boleh beda dari AI jika berargumen.
4. **Uji** — airplane mode, dirty flag, sync.
5. **Refactor** — rapikan repository dan provider.
6. **Jelaskan** — mampu menerangkan tanpa AI.

Uji minimal

- airplane mode penuh
- field hilang di `fromMap`
- badge dirty sebelum/sesudah sync
- `flutter analyze` + `flutter test`

Aktivitas Kelompok

Code Review 15 Menit

Cari minimal:

- 2 potensi bug mapping (`fromMap` / `toMap`)
- 2 masalah sync (dirty tak pernah nol, konflik tanpa aturan)
- 1 masalah arsitektur (SQL di UI)
- 1 masalah UX state (tanpa badge dirty)
- 1 improvement dari rekomendasi AI

Kemudian jawab:

Apakah aplikasi offline ini layak masuk production? Mengapa?

Quiz Cepat

1. Kapan memakai SharedPreferences?

- A. Daftar 1000 catatan
- B. Pengaturan kecil key-value
- C. Query relasional kompleks
- D. Streaming data real-time

2. Fungsi dirty flag?

- A. Mempercantik UI
- B. Menandai data belum tersinkron
- C. Menghapus database
- D. Mempercepat internet

3. Cache-first artinya?

- A. Selalu fetch jaringan dulu
- B. Tampilkan lokal dulu, refresh background
- C. Hapus cache tiap dibuka
- D. Matikan database

Jawaban Quiz

1 → B

SharedPreferences hanya untuk nilai kecil seperti tema dan bahasa.

2 → B

`dirty = 1` = antrean sync yang menunggu koneksi.

3 → B

cache → tampil seketika → refresh → simpan

AI Challenge

Gunakan AI untuk membuat **tabel perbandingan**:

"Bandingkan SharedPreferences, Hive, sqflite, dan Drift untuk Offline Notes + preferensi tema, beserta rekomendasi final."

Kemudian mahasiswa wajib:

1. memahami tiap opsi
2. mencoba instalasi dan skemanya
3. menemukan minimal 2 kelemahan rekomendasi AI
4. memutuskan pilihan final sendiri
5. mendokumentasikan alasan di docs/
6. mencatat perubahan manual

Deliverables

Repository

```
week-05-local-storage-offline-first/  
├── lib/  
├── test/  
├── docs/  
├── README.md  
└── screenshots/
```

README minimal berisi:

- deskripsi aplikasi & storage yang dipakai
- repository/provider yang digunakan
- screenshot offline + badge dirty
- cara menjalankan
- AI tools yang digunakan
- prompt utama + tabel perbandingan
- perubahan manual
- masalah yang ditemukan dan hasil testing

Rubrik Week 5

Komponen	Bobot
SharedPreferences & SQLite	20%
Repository Lokal & Offline-First	25%
State: Loading/Error/Empty	15%
UI & UX	10%
Testing	10%
Code Quality	10%
Responsible AI Usage	10%
Total	100%

Checklist Sebelum Submit

Data

- ☐ Preferensi via SharedPreferences
- ☐ CRUD catatan via SQLite + repository
- ☐ Dirty flag + sync berfungsi
- ☐ Aturan konflik didokumentasikan

UI

- ☐ Berfungsi penuh saat offline
- ☐ Badge dirty akurat
- ☐ Tidak ada query di widget

AI

- ☐ Prompt dicatat & tabel perbandingan ada
- ☐ Ada keputusan final + alasan
- ☐ Kode dipahami mahasiswa

Exit Ticket

Jawab sebelum kelas berakhir:

1. Apa perbedaan:

SharedPreferences
vs
SQLite

2. Apa fungsi:

Cache-first
vs
Dirty flag

3. Sebutkan 3 mekanisme offline-first beserta contohnya.

4. Bagian mana rekomendasi AI yang Anda tolak, dan mengapa?

Key Takeaways

Storage

Pilih storage sesuai kebutuhan, bukan kebiasaan.

Arsitektur

Repository memisahkan UI dari database.

Offline-First

Baca-tulis selalu bisa, sync menyusul.

AI

AI mengusulkan opsi storage.

Developer membandingkan dan memutuskan.

Next Week

Week 6: Authentication, Security & FCM

Offline Notes



Login (Firebase Auth / JWT)



Token aman (secure storage)



Notifikasi real-time (FCM)

Target

Aplikasi yang personal, aman, dan berkomunikasi real-time dengan pengguna.

Build → Verify → Explain

Prinsip Pembelajaran Mobile

```
graph TD; AI --> Generate; Generate --> Developer; Developer --> Verify; Verify --> Test; Test --> Refactor; Refactor --> Explain;
```

Jangan hanya bisa menyimpan data.

Jadilah developer yang memahami alur data yang disimpan.